PUC Minas

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Lista de Exercícios para Preparação para a Primeira Prova

Professora: *Marta Noronha*

Disciplina: *Algoritmos e Estrutura de Dados II*

1. Contagem de Operações e logaritmos

01. Quantas operações de multiplicação são realizadas nos códigos abaixo:

a)
```
for(int i = 0; i < n, i++)
    b = b * 2;
```

b)
```
for(int i = 0; i ≤ n, i++)
    b = b * 2;
```

c)
```
for(int i = 0; i < n + 1, i++)
    b = b * 2;
```

d)
```
for(int i = 1; i < n + 1, i++)
    b = b * 2;
```

e)
```
for(int i = 0; i < n, i++){
    b = b * 2;
    for(int j = 0; j < n, j++){
        b* = a;
    }
}
```

f)
```
for(int i = 0; i ≤ n, i++){
    b = b * 2;
    for(int j = 0; j < m, j++){
        b* = a;
    }
}
```

g)
```
for(int i = 3; i ≤ n, i++){
    b = b * 2;
    b* = a;
    a++;
    for(int j = 0; j ≤ n, j++){
        c* = b;
        b--;
    }
}
```

h)
```
for(int i = n; i > 0, i--)
    b = b * 2;
```

i)
```
for(int i = n; i ≥ 0, i--)
    b = b * 2;
```

j)
```
for(int i = n + 1; i > 0, i--)
    b = b * 2;
```

k)
```
for(int i = n + 1; i ≥ 0, i--)
    b = b * 2;
```

l)
```
for(int i = n; i > 0, i--){
    b = b * 2;
    for(int j = n + 1; j > 0, j--){
        b* = 2;
        c* = b;
    }
}
```

m)
```
for(int i = 0; i ≤ n, i++){
    for(int j = n; j ≥ 0, j--){
        b = b * 2;
    }
}
```

n)
```
for(int i = 0; i < n; i++){
    for(int j = i; j < n; j++){
        a * = 2;
    }
}
```

02. Quantas operações de subtração são realizadas nos códigos abaixo:

a)
```
for(int i = n; i > 0, i/ = 2)
    a− = 2;
```

b)
```
for(int i = 1; i ≤ n, i* = 2)
    c = d − a;
```

c)
```
for(int i = 1; i < n, i* = 2)
    c = d − a;
```

d)
```
for(int i = 1; i < n, i* = 2){
    a− = 5;
    for(int j = n; j > 0, j/ = 2){
        b− = c − d;
    }
}
```

03. Escreva o resultado das funções logarítmicas abaixo:

a) $log_2 512$
b) $\lfloor log_2 512 \rfloor$
c) $\lceil log_2 512 \rceil$
d) $log_2 511$
e) $\lfloor log_2 511 \rfloor$
f) $\lceil log_2 511 \rceil$
g) $log_2 513$
h) $\lfloor log_2 513 \rfloor$
i) $\lceil log_2 513 \rceil$
j) $log_2 1025$
k) $\lfloor log_2 2047 \rfloor$
l) $\lceil log_2 2047 \rceil$

2. Somatórios

Refaça atentamente os exemplos de somatórios apresentados nos slides, disponível no Canvas → página inicial → módulo "Unidade 2" → arquivo "unidade02_somatorio.pdf". Estes **não precisam ser entregues** mas fazê-los irá auxiliar na compreensão da matéria.

Exemplo de resolução de um somatório

Considere o somatório:

$$S_n = \sum_{i=0}^{n} (i-1)^2 \tag{1}$$

Primeiro passo: Desenvolva o produto notável.

$$S_n = \sum_{i=0}^{n} (i^2 - 2i + 1) \tag{2}$$

Perceba que na Equação 2, temos na verdade:

$$S_n = \sum_{i=0}^{n} (i^2 - 2 * i^1 + 1 * i^0) \tag{3}$$

Sabemos que a resolução deste somatório utilizando

$$Sn + a_{(n+1)} = a_0 + \textstyle\sum_{i=0}^{n} a_{i+1}$$

irá resultar em erro porque encontraremos S_n nos dois lados da Equação, sendo cancelados portanto. Logo devemos utilizar um grau maior sobre os termos do polinômio $i^2 - 2i^1 + 1i^0$ deste somatório para chegar ao resultado correto. Teremos então S_m:

$$S_m = \sum_{i=0}^{n} (i^3 - 2i^2 + i^1) \tag{4}$$

Segundo passo: Resolveremos então a equação por partes, começando do polinômio de menor grau:

- $\sum_{i=0}^{n} i^3$
- $2 * \sum_{i=0}^{n} i^2$ (Obs: aplicação da distributividade para calcular o somatório de forma isolada)
- $\sum_{i=0}^{n} i$

Resolvendo o somatório $Sm = \sum_{i=0}^{n} i$

Partindo da Equação $Sm + a_{(n+1)} = a_0 + \sum_{i=0}^{n} a_{(i+1)}$, os termos $a_0 = 0$, $a_{n+1} = (n+1)$ e $a_{(i+1)} = (i+1)$. Lembrando que estamos buscando, ao desenvolver o somatório, $Sn = \sum_{i=0}^{n} 1$.

$$Sm + (n+1) = 0 + \textstyle\sum_{i=0}^{n} (i+1)$$
$$Sm + (n+1) = \textstyle\sum_{i=0}^{n} (i+1)$$
$$Sm + (n+1) = \textstyle\sum_{i=0}^{n} i + \sum_{i=0}^{n} 1$$
$$Sm + (n+1) = Sm + Sn$$
$$Sm - Sm + n + 1 = Sn$$
$$Sn = n + 1$$

Portanto o somatório:

$$\sum_{i=0}^{n} 1 = n + 1 \tag{5}$$

Resolvendo o somatório $Sm = \sum_{i=0}^{n} i^2$

Partindo da Equação $Sm + a_{(n+1)} = a_0 + \sum_{i=0}^{n} a_{(i+1)}$, os termos $a_0 = 0$, $a_{n+1} = (n+1)^2$ e $a_{(i+1)} = (i+1)^2$.

Lembrando que estamos buscando, ao desenvolver o somatório, $Sn = \sum_{i=0}^{n} i$.

$$Sm + (n+1)^2 = 0 + \sum_{i=0}^{n}(i+1)^2$$
$$Sm + (n+1)^2 = \sum_{i=0}^{n}(i+1)^2$$
$$Sm + (n+1)^2 = \sum_{i=0}^{n} i^2 + 2 * \sum_{i=0}^{n} i + \sum_{i=0}^{n} 1$$
$$Sm + (n+1)^2 = Sm + 2 * Sn + \sum_{i=0}^{n} 1$$

O resultado do somatório $\sum_{i=0}^{n} 1$ foi conhecido na Equação 5, sendo ele $\sum_{i=0}^{n} 1 = n + 1$. Portanto, o substituíremos no desenvolvimento:

$$Sm - Sm + (n+1)^2 = 2 * Sn + (n+1)$$
$$2 * Sn = (n+1)^2 - (n+1)$$
$$2 * Sn = n^2 + 2n + 1 - n - 1$$
$$2 * Sn = n^2 + n$$
$$Sn = \frac{n^2+n}{2}$$
$$Sn = \frac{n(n+1)}{2}$$

Portanto o somatório:

$$\sum_{i=0}^{n} i = \frac{n(n+1)}{2} \tag{6}$$

Por fim, resolvendo o somatório $Sm = \sum_{i=0}^{n} i^3$

Partindo da Equação $Sm + a_{(n+1)} = a_0 + \sum_{i=0}^{n} a_{(i+1)}$, os termos $a_0 = 0$, $a_{n+1} = (n+1)^3$ e $a_{(i+1)} = (i+1)^3$.

Lembrando que estamos buscando, ao desenvolver o somatório, $Sn = \sum_{i=0}^{n} i^2$.

$$Sm + (n+1)^3 = 0 + \sum_{i=0}^{n}(i+1)^3$$
$$Sm + (n+1)^3 = \sum_{i=0}^{n}(i+1)^3$$
$$Sm + (n+1)^3 = \sum_{i=0}^{n} i^3 + 3 * \sum_{i=0}^{n} i^2 + 3 * \sum_{i=0}^{n} i + \sum_{i=0}^{n} 1$$
$$Sm + (n+1)^3 = Sm + 3 * Sn + 3 * \sum_{i=0}^{n} i + \sum_{i=0}^{n} 1$$

O resultado do somatório $\sum_{i=0}^{n} 1$ foi conhecido na Equação 5, sendo ele $\sum_{i=0}^{n} 1 = n + 1$. Enquanto o somatório $\sum_{i=0}^{n} i$ foi conhecido na Equação 6, sendo ele $\sum_{i=0}^{n} i = \frac{n(n+1)}{2}$. Portanto, os substituíremos no desenvolvimento:

$$Sm - Sm + (n+1)^3 = 3 * Sn + 3 * \frac{n(n+1)}{2} + (n+1)$$
$$(n+1)^3 = \frac{6*Sn+3*n(n+1)+2(n+1)}{2}$$
$$2(n+1)^3 = 6 * Sn + 3 * n(n+1) + 2(n+1)$$
$$6 * Sn = 2(n+1)^3 - 3n(n+1) - 2(n+1)$$

$$6 * Sn = 2(n^3 + 3n^2 + 3n + 1) - 3n^2 - 3n - 2(n + 1)$$
$$6 * Sn = 2n^3 + 6n^2 + 6n + 2 - 3n^2 - 3n - 2n + 2$$
$$6 * Sn = 2n^3 + 3n^2 + n$$
$$Sn = \frac{2n^3+3n^2+n}{6}$$
$$Sn = \frac{n(n+1)(2n+1)}{6}$$

Portanto o somatório:

$$\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \tag{7}$$

Finalmente, retornemos à Equação 3, dada por $S_n = \sum_{i=0}^{n}(i^2 - 2i + 1)$ e fazendo as substituições dos somatórios encontrados nas Equações 7, 6 e 5, temos:

$$S_n = \sum_{i=0}^{n}(i^2 - 2i + 1)$$
$$S_n = \sum_{i=0}^{n} i^2 - 2 * \sum_{i=0}^{n} i^2 + \sum_{i=0}^{n} 1)$$
$$S_n = \frac{n(n+1)(2n+1)}{6} - 2 * \frac{n(n+1)}{2} + (n + 1)$$
$$S_n = \frac{2n^3+3n^2+n}{6} - n^2 - n + (n + 1)$$
$$S_n = \frac{2n^3+3n^2+n-6n^2-6n+6n+6}{6}$$
$$S_n = \frac{2n^3-3n^2+n+6}{6}$$

Portanto,

$$\sum_{i=0}^{n}(i - 1)^2 = \frac{2n^3 - 3n^2 + n + 6}{6} \tag{8}$$

Prova por indução da validade da fórmula fechada

Após encontrar a fórmula fechada de um somatório, é necessário provar que ela é válida para qualquer valor de n. Uma forma de provar é utilizando o método da indução, onde se parte do princípio que a fórmula é válida para a base e testa-se em seguida, por meio da indução, se será válida para qualquer que seja o valor de n.

Considerando ainda o problema anterior onde têm-se:

$$S_n = \sum_{i=0}^{n}(i - 1)^2 = \frac{2n^3-3n^2+n+6}{6}$$

devemos provar que a Equação é válida para a base. Substituindo $i = \textit{valor da base}$, temos que para o valor da base com $i = 0$:

$$S_{(0)} = (0 - 1)^2 = 1 \tag{9}$$

É necessário validar a base na fórmula fechada encontrada na Equação $\frac{2n^3-3n^2+n+6}{6}$, substituindo qualquer valor n na Equação pelo valor encontrado na base do somatório, ou seja, $n = 0$.

$$S_{(0)} = \frac{2 * 0^3 - 3 * 0^2 + 0 + 6}{6} = \frac{6}{6} = 1 \tag{10}$$

Os valores obtidos substituindo $i = \textit{valor da base}$ nas Equações 9 e 10 são iguais. Se fossem diferentes poderia-se concluir que a fórmula não é válida.

Como a fórmula é válida para a base, deve-se iniciar o processo indutivo utilizando a fórmula mostrada na Equação 8, para provar que a Equação é válida para qualquer valor de n.
Para realizar o processo indutivo, deve-se lembrar que o mesmo será dado por:

$$S_n = S_{n-1} + a_n \tag{11}$$

Portanto cada n presente na Equação 8 deve ser substituído por $n-1$.

$$S_{n-1} = \frac{2(n-1)^3 - 3(n-1)^2 + (n-1) + 6}{6} \tag{12}$$

O último termo, a_n, deve ser calculado com base no enésimo termo do somatório abaixo:

$$S_n = \sum_{i=0}^{n} (i-1)^2$$

Substituindo $i = n$;

$$a_n = (n-1)^2 = n^2 - 2n + 1 \tag{13}$$

Substituindo os valores S_{n-1} e a_n na Equação 11, têm-se:

$$S_n = \frac{2(n-1)^3 - 3(n-1)^2 + (n-1) + 6}{6} + n^2 - 2n + 1$$

$$S_n = \frac{2(n^3 - 3n^2 + 3n - 1) - 3(n^2 - 2n + 1) + (n-1) + 6 + 6n^2 - 12n + 6}{6}$$

$$S_n = \frac{2n^3 - 6n^2 + 6n - 2 - 3n^2 + 6n - 3 + n - 1 + 6 + 6n^2 - 12n + 6}{6}$$

$$S_n = \frac{2n^3 - 3n^2 + n + 6}{6} \text{ c.q.d.}$$

Como a Equação encontrada por indução é idêntica à Equação 8, então está provado que a fórmula fechada é válida para qualquer valor de n.

Exercícios

01. Encontre a fórmula fechada de $\sum_{i=1}^{n} (4i^2 - 1)$. Prove a validade da fórmula fechada por indução.

02. Encontre a fórmula fechada de $\sum_{i=1}^{n} (2i - 1)^2$. Prove a validade da fórmula fechada por indução.

03. Encontre a fórmula fechada de $\sum_{i=1}^{n} (3i - 2)^2$. Prove a validade da fórmula fechada por indução.

04. Encontre a fórmula fechada de $\sum_{i=1}^{n} [i^2(2i + 1)]$. Prove a validade da fórmula fechada por indução.

05. Prove por indução matemática que $\sum_{i=1}^{n} (2i - 1) = 1 + 3 + 5 + \cdots + (2n - 1) = n^2, \forall n \geq 0, n \in \mathbb{N}$.

06. Prove por indução matemática que $\sum_{i=1}^{i=n} \frac{1}{i(i+1)} = \frac{1}{1*2} + \frac{1}{2*3} + \cdots + \frac{1}{n*(n+1)} = \frac{n}{n+1}, \forall n \geq 1, n \in \mathbb{N}$.

07. Prove por indução matemática que $\sum_{i=1}^{i=n} i(i + 3) = \frac{1}{3}n(n + 1)(n + 5), \forall n \geq 1, n \in \mathbb{N}$.

08. Prove por indução matemática que $\sum_{i=1}^{i=n} i^2(i - 1) = \frac{1}{12}n(n - 1)(n + 1)(3n + 2), \forall n \geq 2, n \in \mathbb{N}$.

09. Prove por indução matemática que $\sum_{i=1}^{i=n} (3^i) = \frac{3}{2}(3^n - 1), \forall n \geq 1, n \in \mathbb{N}$.

10. Escreva o significado dos seguintes somatórios (obs: não precisa calcular o valor final):

EXEMPLO: $\sum_{i=0}^{8} i = 0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8$

a) $\sum_{i=1}^{7} i^4$

b) $\sum_{i=0}^{4} 3^{i+1}$

c) $\sum_{i=1}^{4} x_i^2$

d) $\sum_{i=1}^{5} \frac{(-1)^i}{2*(i+1)+i}$

e) $\sum_{i=1}^{6} (-2)^r i^2$

11. Expresse cada expressão na notação de somatório:

EXEMPLO: $0+1+2+3+4+5+6+7+8 = \sum_{i=0}^{8} i$

a) $1+2+4+8+16+32+64+128$

b) $2+4+6+8+10+12+14+16+18$

c) $\frac{1}{1}+\frac{1}{2}+\frac{1}{3}+\frac{1}{4}+\frac{1}{5}$

d) $\frac{x}{1}+\frac{x}{4}+\frac{x}{9}+\frac{x}{16}+\frac{x}{25}$

e) $1+4+27+256+3.125+46.656$

f) $1+4+7+10+13+16$

g) $3+12+33+90+255+744+\cdots+a_{n-1}+a_n$

3. Complexidade

01. Dado cada trecho de código abaixo, determine, em função de n (e de m ou k, quando presentes),o número de vezes que a operação de multiplicação é executada. Em seguida, expresse a complexidade assintótica do algoritmo utilizando as notações $\mathcal{O}$, Ω e Θ.

a)
```
for(int i = 0; i < n; i++)
    a* = 2;
```

b)
```
for(int i = 0;i ≤ n; i++)
    a* = 2;
```

c)
```
for(int i = 0;i ≤ n; i++){
    a* = 2;
    for(int j = 0;j < m; j++){
        b* = a;
    }
}
```

d)
```
for(int i = n+1; i > 0; i--)
    a* = 2;
```

e)
```
for(int i = 0;i ≤ n; i++){
    for(int j = n;j ≥ 0; j--){
        a* = 2;
    }
}
```

f)
```
for(int i = 0;i ≤ n; i++){
    for(int j = n;j ≥ 0; j--){
        for(int k = 0;k ≤ n; k++){
            a* = 2;
        }
    }
}
```

g)
```
for(int i = 0; i ≤ n, i++){
    a* = 2;
    for(int j = n; j ≥ 0, j--){
        for(int k = 0; k ≤ n, k++){
            b* = 2a;
        }
    }
}
```

02. Dado cada trecho de código abaixo, analise a complexidade assintótica considerando as operações de subtração (por exemplo, `a-=2` e `b-=1`). Para cada caso, determine, em função de n (e de quaisquer outras variáveis, se presentes), quantas vezes cada operação de subtração é realizada e, a partir dessa contagem, expresse a complexidade utilizando as notações $\mathcal{O}$, Ω e Θ.

a)
```
for(int i = n; i > 0; i/ = 2)
    a− = 2;
```

b)
```
for(int i = 1; i ≤ n; i* = 2)
    a− = 2;
```

c)
```
for(int i = 1; i < n; i* = 2){
    a− = 2;
    for(int j = n; j > 0; j/ = 2){
        b− = 1;
        c = b − 1;
    }
}
```

03. Uma função $g(n) = \mathcal{O}(f(n))$ se existirem as constantes positivas c e m tais que, para $n \geq m$, temos que $|g(n)| \leq c \times |f(n)|$. Qual a complexidade, $\mathcal{O}(f(n))$, de um algoritmo descrito pela função $g(x) = n^2 + 4n + 1$? Prove a complexidade apresentando os valores de c e de m.

04. Considere agora a função $g(n) = 3n + 5$. Utilize a definição de notação $\mathcal{O}$ para demonstrar que $g(n) = \mathcal{O}(n)$. Determine valores apropriados para as constantes c e m.

05. Seja $g(n) = 5\log_2(n) + 10$. Prove que $g(n) = \mathcal{O}(\log n)$ encontrando constantes c e m que satisfaçam a definição.

06. Dada a função $g(n) = 2^n + n$, mostre que $g(n) = \mathcal{O}(2^n)$. Identifique os valores de c e m para os quais a desigualdade $|g(n)| \leq c \times |2^n|$ vale para todo $n \geq m$.

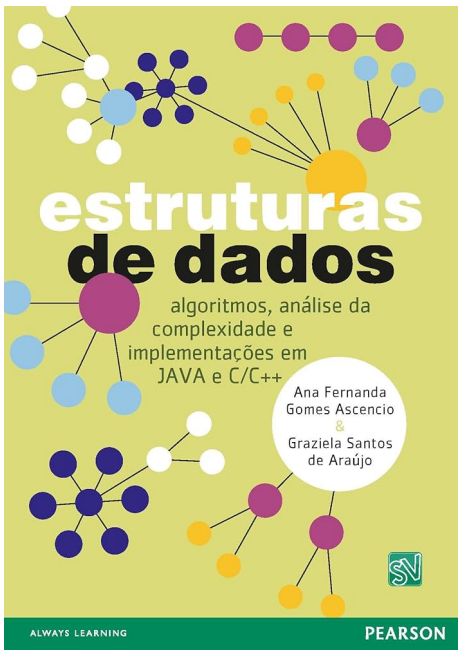
07. Considere $g(n) = n^3 - 2n^2 + 7$. Utilizando a definição de notação assintótica, demonstre que $g(n) = \mathcal{O}(n^3)$, indicando explicitamente os valores de c e m.

08. Considere $g(n) = 7n - 9$. Prove que $g(n) = \Theta(n)$ mostrando tanto que $g(n) = \mathcal{O}(n)$ quanto que $g(n) = \Omega(n)$, e encontre os respectivos valores de c (ou c_1 e c_2) e m, onde $m = max\{m1, m2\}$.

4. ALGORITMOS DE ORDENAÇÃO

Indicação de Leitura (disponível na biblioteca em formato físico e *online*):

- ASCENCIO, Ana Fernanda Gomes; ARAÚJO, Graziela Santos de. Estruturas de dados: algoritmos, análise da complexidade e implementações em Java e C/C++. São Paulo: Pearson Prentice Hall, c2010. xii, 432 p. ISBN 9788576058816.



Neste material, apresento o passo a passo da execução dos algoritmos da bolha (*bubble sort*), de inserção (*insertion sort*) e de seleção (*selection sort*). Para compreender o funcionamento de outros algoritmos avançados, como *quicksort*, *heapsort* e *mergesort*, consulte o livro citado acima, onde esses métodos são detalhados por meio de figuras e explicações aprofundadas.

4.1. Algoritmo da bolha

A ordenação pelo método da bolha consiste em “borbulhar" os elementos até que eles estejam em sua posição correta em um vetor, ou seja, ele compara o elemento considerado como “bolha" com os demais elementos do vetor até encontrar a posição em que este deve ficar. Considere o algoritmo mostrado na Figura 1 que ordena vetores em ordem crescente:

```
void bubblesort() {
  for (int i = (n - 1); i > 0; i--) {
    for (int j = 0; j < i; j++) {
      if (array[j] > array[j + 1]) {
        swap(j, j+1);
      }
    }
  }
}
```

Figure 1: Algoritmo de ordenação por bolha (*BubbleSort*)

O algoritmo mostrado na Figura 1 “borbulha" o maior valor encontrado no vetor para a posição final.

Quando o contador i decrementa, sabemos que os elementos após i já estão ordenados, portanto o contador j não é comparado com os elementos contidos no vetor após o índice marcado por i. Quando o elemento que está contido no vetor na posição j é maior do que o elemento da posição $j+1$ então é feita a troca de posição entre os elementos.

Considerando que a comparação sempre será realizada podemos construir o seguinte somatório para obter a fórmula fechada para conhecer a complexidade em termos de quantidade de comparações. Temos que o contador i itera de 1 até $n-1$, enquanto o contador j itera de 0 até $i-1$. Para cada iteração temos 1 comparação. Temos então o seguinte somatório:

$$\sum_{i=1}^{i=n-1} \sum_{j=0}^{j=i-1} 1 \tag{14}$$

que pode ser resolvido como:

$$\sum_{i=1}^{i=n-1} \sum_{j=0}^{j=i-1} 1 = \sum_{i=1}^{i=n-1} 1 * (\underbrace{i-1}_{\text{Lim. Sup.}} - \underbrace{0}_{\text{Lim. Inf.}} + 1) = \sum_{i=1}^{i=n-1} i \tag{15}$$

onde (termos coloridos se anulam)[1]:

$$\sum_{i=1}^{i=n-1} i = \frac{(n-1-1+1)*((n-1-1+1)+1)}{2} = \frac{(n-1)*n}{2} \tag{16}$$

que equivale a:

$$\frac{(n^2-n)}{2} \textit{ comparações.} \tag{17}$$

Como o algoritmo analisa posições que foram analisados em etapas anteriores, com exceção da parte do vetor que já está ordenada então o algoritmo executa $\frac{n^2-n}{2}$ comparações nos 3 casos (pior, médio e melhor caso) conforme visto na fórmula fechada apresentada em (17), portanto a sua complexidade em relação a quantidade de comparações é da ordem de $\Theta(n^2)$, ou seja, sua complexidade de classe é quadrática.

Em relação ao número de movimentações, consideremos o pior caso onde os elementos do vetor estão em ordem decrescente. Portanto em cada iteração serão feitas 3 movimentações de elementos nos vetores. Portanto temos o total de movimentações é dado pela Equação 18, onde o mesmo também é da ordem de $\Theta(n^2)$.

$$\sum_{i=1}^{i=n-1} \sum_{j=0}^{j=i-1} 3 = 3 * \frac{n(n-1)}{2} \tag{18}$$

Para melhor compreensão do funcionamento do algoritmo, analise a ordenação do vetor [10,6,3,4,2] (Figura 2) que foi feita seguindo os passos do algoritmo apresentado na Figura 1.

[1] Já mostramos em sala que o somatório de $\sum_{i=1}^{i=n} i$ corresponde ao somatório de Gauss onde a fórmula fechada é igual a $\frac{n*(n+1)}{2}$, o qual utilizamos para resolver esse somatório.

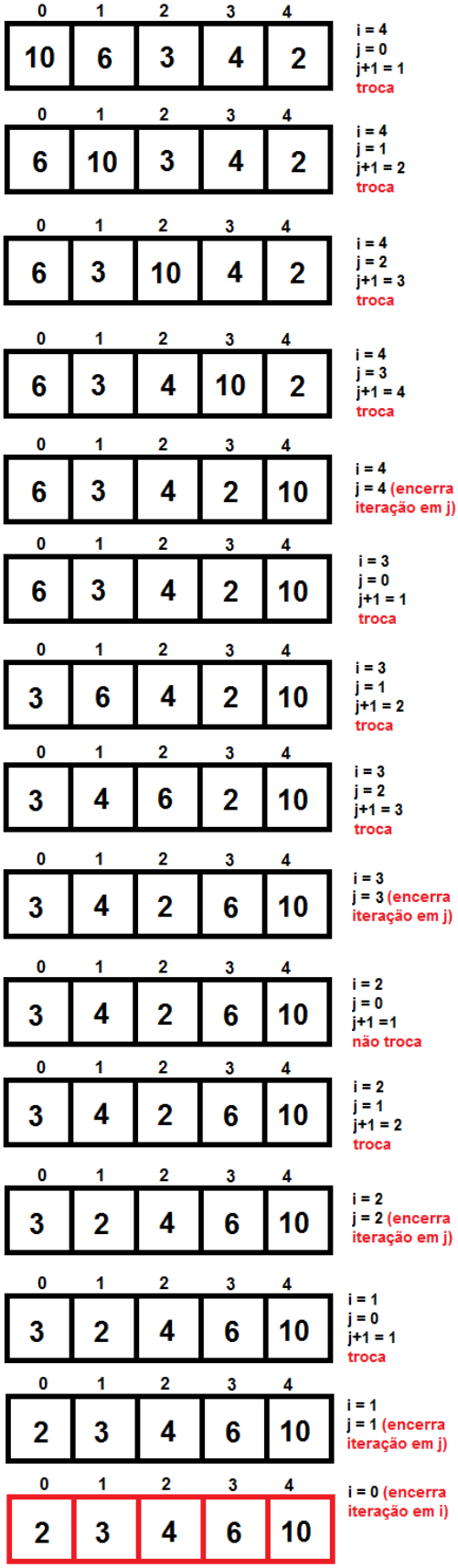


Figure 2: Ordenação do vetor [10,6,3,4,2] por algoritmo de bolha

4.2. Algoritmo de seleção

O algoritmo de seleção mostrado na Figura 3 busca pelo índice do menor elemento do vetor para posteriormente inserir o menor elemento em sua posição correta. Somente após verificar todas as posições do vetor que será realizada a troca do elemento que está na posição atual do iterador de i com o menor elemento. Essa troca será realizada se $i! = menor$.

```
void selectionsort() {
    for (int i = 0; i < (n - 1); i++) {
      int menor = i;
      for (int j = (i + 1); j < n; j++){
          if (array[menor] > array[j]){
            menor = j;
          }
      }

      if(i != menor){
         int aux = array[menor];
         array[menor] = array[i];
         array[i] = aux;
      }
    }
}
```

Figure 3: Algoritmo de ordenação por seleção (*Selection Sort*)

Temos o número de comparações do tipo $array[menor] > array[j]$ dada pela Equação 19 para o melhor, médio e pior caso.

$$\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \tag{19}$$

Em relação as movimentações, no pior caso quando o array está ordenado em ordem decrescente, serão feitas 3 movimentações por iteração externa do iterador i. Como i itera de 0 até n-2 então podemos ter 3*(n-1) movimentações no pior caso. Portanto em relação ao número de movimentações, o algoritmo de seleção tem uma complexidade linear ($\Theta(n)$).

Para melhor compreensão do funcionamento do algoritmo, analise a ordenação do vetor [10,6,3,4,2] (Figura 4) que foi feita seguindo os passos do algoritmo apresentado na Figura 3.

0	1	2	3	4
10	6	3	4	2

i = 0, i < 4, j = 1, j < 5
10 > 6 V
menor = 1

i = 0, i < 4, j = 2, j < 5
6 > 3 V
menor = 2

i = 0, i < 4, j = 3, j < 5
3 > 4 F

i = 0, i < 4, j = 4, j < 5
4 > 2 V
menor = 4

i = 0, i < 4, j = 5, j < 5 F (encerra)

troca(array[4],array[0])

0	1	2	3	4
2	6	3	4	10

i = 1, i < 4, j = 2, j < 5
6 > 3 V
menor = 2

i = 1, i < 4, j = 3, j < 5
3 > 4 F

i = 1, i < 4, j = 4, j < 5
3 > 10 F

i = 1, i < 4, j = 5, j < 5 F (encerra)

troca(array[2],array[1])

0	1	2	3	4
2	3	6	4	10

i = 2, i < 4, j = 3, j < 5
6 > 4 V
menor = 3

i = 2, i < 4, j = 4, j < 5
4 > 10 F

i = 2, i < 4, j = 5, j < 5 F (encerra)

troca(array[3],array[2])

0	1	2	3	4
2	3	4	6	10

i = 3, i < 4, j = 4, j < 5
6 > 10 F

i = 0, i < 4, j = 5, j < 5 F (encerra)

i= 4, i < 4 F (encerra)

Figure 4: Ordenação do vetor [10,6,3,4,2] por algoritmo de seleção

4.3. Algoritmo de inserção

O algoritmo de ordenação por inserção ordena os elementos ao considerar que existe uma parte ordenada e outra desordenada no vetor. O algoritmo itera selecionando elementos na parte que está desordenada e os insere em sua posição correta na parte ordenada. Para isso um elemento é selecionado na parte desordenada do vetor e seu valor é salvo em uma variável temporária. Esse elemento salvo será comparado com os elementos que estão na parte ordenada. Alguns elementos da parte ordenada poderão ser deslocados até que se ache a posição correta do elemento que foi salvo. O algoritmo continua a iteração até que todo o vetor esteja ordenado.

```
void insertionsort() {
  for (int i = 1; i < n; i++) {
      int tmp = array[i];
      int j = i - 1;

      while ((j >= 0) && (array[j] > tmp)) {
        array[j + 1] = array[j];
        j--;
      }
      array[j + 1] = tmp;
  }
}
```

Figure 5: Algoritmo de ordenação por inserção (*Insertion Sort*)

Em relação a complexidade do algoritmo sabemos que o laço externo é iterado (n-1) vezes usando o contador i. Já o laço interno relacionado ao contador j é executado $i-1$ vezes dependendo do caso. No melhor caso teremos que a comparação $array[j] > temp$ será sempre falsa porque o vetor está ordenado em ordem crescente, portanto não será feita nenhuma iteração do contador j. Para esse caso teremos realizado somente (n-1) comparações. No pior caso, quando o vetor está ordenado em ordem decrescente temos que executar sempre os laços internos, sendo que nesse caso o número de iterações de j depende do valor de i. Nesse caso a complexidade em relação ao número de comparações é obtida pelo somatório mostrado na Equação 21.

$$C_i(n) = \sum_{i=1}^{i=n-1} \sum_{j=0}^{j=i-1} 1 = \frac{n*(n-1)}{2} = \frac{(n^2-n)}{2} \tag{20}$$

Logo a complexidade é $\Theta(n^2)$, ou seja, da classe quadrática.

Em relação ao número de movimentações, temos duas movimentações no laço externo que serão repetidas (n-1) vezes sendo elas:

- int tmp = array[i];
- array[j + 1] = tmp;

mais uma movimentação no laço interno caso a comparação $(j >= 0)\&\&(array[j] > tmp)$ seja verdadeira, sendo ela:

- array[j + 1] = array[j];

Podemos notar que o número de movimentações no laço interno é igual ao número de comparações (Equação 21) menos um, dado que o contador de j itera a partir de $i-1$. Logo o número de movimentações no melhor caso é dado pela quantidade de movimentações internas mais 2 externas, ou seja:

$$M_i(n) = (C_i(n) - 1) + 2 = C_i(n) + 1 \tag{21}$$

Considerando que no melhor caso, temos somente 1 comparação então $C_i(n) = 1$ para cada valor de i. Portanto $M_i(n) = C_i(n) + 1 = 2$ para cada i. Como i itera $(n-1)$ vezes então temos:

$$M(n) = \sum_{i=1}^{n-1} 2 = 2(n-1) \tag{22}$$

Logo o número de movimentações no melhor caso é da ordem de $\Theta(n)$. Portanto teremos um custo aproximadamente linear se o vetor não estiver muito desordenado.

Já o número de movimentações internas no pior caso é dado na Equação 21 mais um. Sabendo que M(n) segue a ordem $M(n) = 2, 3, 4, ..., (n-2)+1, (n-1)+1$ então podemos reescrever essa função como $\sum_{i=2}^{n} i$. Fazendo mudança dos limites podemos reescrever como $\sum_{i=0}^{n-2} i + 2$.

$$M(n) = \sum_{i=0}^{n-2} i + 2 = \sum_{i=0}^{n-2} i + \sum_{i=0}^{n-2} 2 = \frac{(n-1)(n-2)}{2} + 2 * (n-1) = \frac{n^2 + n - 2}{2} \tag{23}$$

Consulte os slides 132 a 138 do algoritmo de inserção disponível no Canvas para detalhes.

Para melhor compreensão do funcionamento do algoritmo, analise a ordenação do vetor [10,6,3,4,2] (Figura 6) que foi feita seguindo os passos do algoritmo apresentado na Figura 5 .

0	1	2	3	4	
10	6	3	4	2	i = 1, 1 < 5, tmp = 6, j = 0 0>=0 && 10 > 6 V
10	10	3	4	2	tmp = 6, j = -1 -1>=0 F
6	10	3	4	2	i = 2, 2 < 5, tmp = 3, j = 1 1>=0 && 10 > 3 V
6	10	10	4	2	tmp = 3, j = 0 0>=0 && 6 > 3 V
6	6	10	4	2	tmp = 3, j = -1 -1>=0 F
3	6	10	4	2	i = 3, 3 < 5, tmp = 4, j = 2 2>=0 && 10 > 4 V
3	6	10	10	2	tmp = 4, j = 1 1>=0 && 6 > 4 V
3	6	6	10	2	tmp = 4, j = 0 0>=0 && 3 > 4 F
3	4	6	10	2	i = 4, 4 < 5, tmp = 2, j = 3 3>=0 && 10 > 2 V
3	4	6	10	10	tmp = 2, j = 2 2>=0 && 6 > 2 V
3	4	6	6	10	tmp = 2, j = 1 1>=0 && 4 > 2 V
3	4	4	6	10	tmp = 2, j = 0 0>=0 && 3 > 2 V
3	3	4	6	10	tmp = 2, j = -1 -1>=0 F
2	3	4	6	10	Vetor ordenado

Figure 6: Ordenação do vetor [10,6,3,4,2] por algoritmo de inserção

EXERCÍCIOS

Obs: Para os exercícios 1 a 3 não é necessário mostrar a ordenação por meio de figuras. Somente mostre a(s) parte(s) que deve(m) ser alterada(s) no código.

1) Qual mudança deve ser feita no algoritmo da bolha, implementado na Figura 1, para que a ordenação seja feita na ordem decrescente?

2) Qual mudança deve ser feita no algoritmo de seleção, implementado na Figura 3, para que a ordenação seja feita na ordem decrescente?

3) Qual mudança deve ser feita no algoritmo de inserção, implementado na Figura 5, para que a ordenação seja feita na ordem decrescente?

4) Ordene o vetor abaixo em ordem crescente usando os métodos de bolha, inserção, seleção, mergesort, heapsort e quicksort.

4	7	5	1	3

5) Uma professora mantém uma lista de notas sempre ordenada em ordem crescente. Ao inserir uma nova nota, deseja-se minimizar o número de operações realizadas.

Considerando os algoritmos Insertion Sort, Bubble Sort e Selection Sort:

a) Indique o algoritmo mais adequado para inserir a nova nota.

b) Explique por que ele é mais eficiente nesse cenário.

c) Justifique por que os outros algoritmos não são ideais.

6) Considere a execução de um algoritmo de ordenação sobre um vetor. Os estados intermediários apresentados a seguir correspondem à primeira iteração do algoritmo:

```
[8, 3, 7, 4]
[3, 8, 7, 4]
[3, 7, 8, 4]
[3, 7, 4, 8]
```

a) Identifique qual algoritmo de ordenação foi utilizado.

b) Justifique sua resposta com base no comportamento observado nos estados intermediários.

7) Você foi contratado como consultor para escolher o(s) algoritmo(s) de ordenação mais adequado(s) em diferentes cenários.

Considerando os algoritmos Insertion Sort, Merge Sort, Quick Sort e Heap Sort, responda:

a) Qual algoritmo é mais indicado para dados quase ordenados?

b) Qual algoritmo é mais indicado para grandes volumes de dados considerando o desempenho no caso médio?

c) Qual algoritmo deve ser escolhido quando há necessidade de estabilidade?

d) Qual algoritmo é mais adequado quando se deseja baixo uso de memória?

Justifique cada uma de suas respostas com base nas características dos algoritmos.

8) Considere os algoritmos abaixo:

Bubble Sort

```java
public static void bubble(int[] array) {
    for (int i = (array.length - 1); i > 0; i--) {
        for (int j = 0; j < i; j++) {
            if (array[j] > array[j + 1]) {
                swap(array, j, j+1);
            }
        }

        for(int k = 0; k < array.length; k++) {
            System.out.print(array[k] + " ");
        }
        System.out.println();
    }
}
```

Selection Sort

```java
public static void selection(int[] array) {
    for (int i = 0; i < (array.length - 1); i++) {
        int menor = i;
        for (int j = (i + 1); j < array.length; j++){
            if (array[menor] > array[j]){
                menor = j;
            }
        }
        swap(array, menor, i);

        for(int k = 0; k < array.length; k++) {
            System.out.print(array[k] + " ");
        }
        System.out.println();
    }
}
```

Insertion Sort

```java
public static void insertion(int[] array) {
    for (int i = 1; i < array.length; i++) {
        int tmp = array[i];
        int j = i - 1;

        while ((j >= 0) && (array[j] > tmp)) {
            array[j + 1] = array[j];
            j--;
        }
        array[j + 1] = tmp;

        for(int k = 0; k < array.length; k++) {
            System.out.print(array[k] + " ");
        }
        System.out.println();
    }
}
```

Shell Sort

```java
public static void shellsort(int[] array) {
    int h = 1;

    do { h = (h * 3) + 1; } while (h < array.length);

    do {
        h /= 3;
        for(int cor = 0; cor < h; cor++){
            insercaoPorCor(array, cor, h);
        }
    } while (h != 1);
}

public static void insercaoPorCor(int[] array, int cor, int h){
    for (int i = (h + cor); i < array.length; i+=h) {
        int tmp = array[i];
        int j = i - h;
        while ((j >= 0) && (array[j] > tmp)) {
            array[j + h] = array[j];
            j-=h;
        }
        array[j + h] = tmp;

        for(int k = 0; k < array.length; k++) {
            System.out.print(array[k] + " ");
        }
        System.out.println();
    }
}
```

Quick Sort

```java
private static void quicksort(int[] array, int esq, int dir) {
    int i = esq, j = dir;
    int pivo = array[(dir+esq)/2];

    while (i <= j) {
        while (array[i] < pivo) i++;
        while (array[j] > pivo) j--;
        if (i <= j) {
            swap(array, i, j);
            i++;
            j--;
        }
    }

    for(int k = 0; k < array.length; k++) {
        System.out.print(array[k] + " ");
    }
    System.out.println();

    if (esq < j) quicksort(array, esq, j);
    if (i < dir) quicksort(array, i, dir);
}
```

Merge Sort

```java
private static void mergesort(int[] array, int esq, int dir) {
    if (esq < dir){
        int meio = (esq + dir) / 2;
        mergesort(array, esq, meio);
        mergesort(array, meio + 1, dir);
        intercalar(array, esq, meio, dir);
    }
}

public static void intercalar(int[] array, int esq, int meio, int dir){
    int n1, n2, i, j, k;

    n1 = meio-esq+1;
    n2 = dir - meio;

    int[] a1 = new int[n1+1];
    int[] a2 = new int[n2+1];

    for(i = 0; i < n1; i++){
        a1[i] = array[esq+i];
    }

    for(j = 0; j < n2; j++){
        a2[j] = array[meio+j+1];
    }

    a1[i] = a2[j] = 0x7FFFFFFF;

    for(i = j = 0, k = esq; k <= dir; k++){
        array[k] = (a1[i] <= a2[j]) ? a1[i++] : a2[j++];
    }

    for(int l = 0; l < array.length; l++) {
        System.out.print(array[l] + " ");
    }
    System.out.println();
}
```

Heap Sort

```java
public static int[] heapsort(int[] array) {
    int[] tmp = new int[array.length+1];
    for(int i = 0; i < array.length; i++){
        tmp[i+1] = array[i];
    }
    array = tmp;

    for(int tamHeap = 2; tamHeap < array.length; tamHeap++){
        construir(array, tamHeap);
    }

    for(int k = 1; k < array.length; k++) {
        System.out.print(array[k] + " ");
    }
    System.out.println();

    int tamHeap = array.length-1;
    while(tamHeap > 1){
        swap(array, 1, tamHeap--);
        reconstruir(array, tamHeap);

        for(int k = 1; k < array.length; k++) {
            System.out.print(array[k] + " ");
        }
        System.out.println();
    }

    tmp = array;
    array = new int[array.length-1];
    for(int i = 0; i < array.length; i++){
        array[i] = tmp[i+1];
    }
    return array;
}

public static void construir(int[] array, int tamHeap){
    for(int i = tamHeap; i > 1 && array[i] > array[i/2]; i /= 2){
        swap(array, i, i/2);
    }
}

public static void reconstruir(int[] array, int tamHeap){
    int i = 1;
    while(i <= (tamHeap/2)){
        int filho = getMaiorFilho(array, i, tamHeap);
        if(array[i] < array[filho]){
            swap(array, i, filho);
            i = filho;
        }else{
            i = tamHeap;
        }
    }
}

public static int getMaiorFilho(int[] array, int i, int tamHeap){
    int filho;
    if (2*i == tamHeap || array[2*i] > array[2*i+1]){
        filho = 2*i;
    } else {
        filho = 2*i + 1;
    }
    return filho;
}
```

Considere o vetor de entrada para cada um dos algoritmos:

[5,1,3,8,6,2]

Utilizando o vetor de entrada acima, qual algoritmo produz as saídas abaixo: (OBS: Considere que a primeira linha foi impressa no programa principal, "*main*".)
a)

5 1 3 8 6 2
1 5 3 8 6 2
1 3 5 8 6 2
1 3 5 8 6 2
1 3 5 6 8 2
1 2 3 5 6 8

Resposta:

b)

5 1 3 8 6 2
1 5 3 8 6 2
1 3 5 8 6 2
1 3 5 6 8 2
1 3 5 2 6 8
1 2 3 5 6 8

Resposta:

c)

5 1 3 8 6 2
1 5 3 8 6 2
1 2 3 8 6 5
1 2 3 8 6 5
1 2 3 5 6 8
1 2 3 5 6 8

Resposta:

d)

5 1 3 8 6 2
2 1 3 8 6 5
1 2 3 8 6 5
1 2 3 5 6 8

Resposta:

e)

5 1 3 8 6 2
1 3 5 6 2 8
1 3 5 2 6 8
1 3 2 5 6 8
1 2 3 5 6 8
1 2 3 5 6 8

Resposta:

f)

5 1 3 8 6 2
8 6 3 1 5 2
6 5 3 1 2 8
5 2 3 1 6 8
3 2 1 5 6 8
2 1 3 5 6 8
1 2 3 5 6 8

g)

5 1 3 8 6 2
5 1 3 8 6 2
5 1 3 8 6 2
1 5 3 8 6 2
1 3 5 8 6 2
1 3 5 8 6 2
1 3 5 6 8 2
1 2 3 5 6 8